Electronics and
Computer Science

UNIVERSITY OF
Southampton

8.1: Procedural blocks

Procedural blocks (processes)

- initial // executes only once at the beginning of simulation
- final // executes once at the end of simulation – very rare
- always, always_comb, always_latch, always_ff
  // these blocks execute continuously when triggered by events
- task // executes when called, similar to function with no return value
- function // executes when called and returns a value

Simple Function

```
function int sqr(int a);
  sqr = a*a;
endfunction
```

In SystemVerilog, we can also write

```
function int sqr(int a);
  return a*a;
endfunction
```

We can also specify the input

```
function int sqr(input int a);
  sqr = a*a;
endfunction
```

This is an example of a "pure" function: no modification of variables outside the function; no output or inout parameters.

(System)Verilog allows "impure" functions, but this is often considered bad practice – difficult to understand and keep track of.

Constant Function

Called once at elaboration (link) time

```
module decoderlogN
  #(parameter N = 8)
  (output logic [N-1:0] y,
    input logic [clog2(N)-1:0] a);

//function clog2 declared here

 always_comb
    y = 1'b1 << a;

endmodule
```

```
function int clog2(input int n);
   begin  // begin .. end aren't
             // strictly necessary
   clog2 = 0;
   n--;
   while (n > 0)
     begin
      clog2++;
      n >>= 1;
    end
   end
endfunction
```

$clog2 is a system function;
clog2 is here as an example

Reentrant (automatic) functions

- In C, C++ etc. functions are "reentrant"
  - New data structure is created each time the function is called.
- In Verilog, by default, functions are not reentrant
  - Data structure is shared between all active instances of a function
- For example, this fails

```
function int factorial (input int op);
  if (op >= 2)
    factorial = factorial (op - 1) * op;
  else
    factorial = 1;
endfunction
```

- We can force a function to be reentrant

```
function automatic int factorial (input int op);
  if (op >= 2)
    factorial = factorial (op - 1) * op;
  else
    factorial = 1;
endfunction
```

This will work correctly – new data structure for each instance

system $functions

System Verilog provides a large number of built-in system commands and tasks

$display - displays a message

$time - returns current simulation time

$finish - terminates simulation

$monitor - displays a message if there are changes in monitored variables

$error - displays an error message in an assertion

$past() - return the value of an expression in the previous clock cycle.

$warning - displays a warning message in an assertion

$fatal - specifies a run-time fatal error

$info - displays an information message in an assertion

$random – returns a random number

etc.

Electronics and
Computer Science

UNIVERSITY OF
Southampton

8.2: Operators

SystemVerilog operators

- Operator groups
  - arithmetic
  - logical
  - relational
  - equality (sometimes included in relational)
  - reduction (bitwise logic)
  - shift
  - concatenation and replication
  - conditional

Arithmetic operators

*	Multiplication
/	Division
+	Addition
-	Subtraction
%	Modulus (remainder)
+	Unary plus
-	Unary minus

- Arithmetic operators will be synthesised to *combinational* logic
  - arithmetic operations + - are mapped into standard adders/subtractors
  - the multiplication operator * can be mapped into embedded multipliers but you have to follow code templates provided by the FPGA vendor
    - mapping * into combinational (cellular) multipliers using standard logic elements is very costly – use embedded multipliers instead.

Logical operators (not to be confused with bitwise/reduction operators)

!	Negation
&&	Logical AND
||	Logical OR

Uses are similar to those in C.
Treat non-zero values (e.g. bits or integers) as logical 1
Zero is false

Relational and equality operators

>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

==	Logical equality
!=	Logical inequality
===	Case equality
!==	Case inequality
==?	Wildcard equality
!=?	Wildcard inequality

Equality operators:

1. == and != result in X if any of their operands contains an X or Z, e.g. X == 1 -> X
2. === and !== check the 4-state logic explicitly therefore, X and Z values shall either match or mismatch, never resulting in X, e.g. X === 1 -> 0 (a mismatch)

3. ==? and != ? treat X or Z as wild cards that match any value, thus, they too never result in X, e.g. X ==? 1 -> 1 (a match)

Bitwise and Reduction operators

~	bitwise negation (binary complement)
&	bitwise AND
~&	bitwise NAND
|	bitwise OR
~|	bitwise NOR
^	bitwise XOR
~^	bitwise XNOR

Bitwise means apply to each bit (of a vector or word) individually.
Reduction operator can be applied to a vector, e.g. to AND together all the bits of the vector to give a single bit result.

Shift operators

>>	logical right shift
<<	logical left shift
>>>	arithmetic right shift
<<<	arithmetic left shift

Uses are similar to those in C which has the logical shifts; distinction between logical and arithmetic shifts was first introduced in Verilog-2001

Logical right shift puts 0 at left end

Arithmetic right shift replicates leftmost bit (i.e. preserves the sign

Logical and Arithmetic left shift both put 0 at right end

Other operators

- Concatenation
  {} e.g. {4'b1001, 2'b11} = 6'b100111
- Replication
  {n{m }} - replicate m for n times, e.g. {8{1'b0}} = 8'b00000000
- Conditional
  ? : e.g. (c==1'b1 ? a : 1'bZ)
- Assignment
  =, +=, -=, *=, /=, %=, &=, ^=, <<=, >>=, <<<=, >>>=
  Notice that these are all *blocking* assignments
- Increment/decrement
  ++,-- use with caution, x++ is equivalent to: x = x+1 not: x <= x+1
  i.e. *blocking* assignments; in sequential logic counters should be implemented using *nonblocking* : count <= count+1

Electronics and
Computer Science

UNIVERSITY OF
Southampton

8.3: Synthesising Combinational Logic

Writing synthesisable code for combinational logic

- when writing code for decoders or multiplexers, use conditional statements
  - if-then-else
  - case
- common mistake:

  conditional statements with incompletely specified signals cannot be mapped into combinational logic:

```
if(a)
   b = 1'b0; // b is incompletely specified
```

  will create an accidental latch
- use **unique** qualifier to ensure completeness checks
  - alternatively in case statements, use the **default** clause (but see later)
  - assign default values to all signals driven by combinational logic blocks
  - pay attention to warnings

Case statements

```
case(opcode)
       NOP, J: ;
       ADD, ADDI: ALUfunc = RADD;
       SUBI, BEQ, BNE, BGE, BLO:
          ALUfunc = RSUB;
       LDI:   ALUfunc = RB;
   endcase
//nothing happens for
//an undefined opcode
```

```
unique case(opcode)
…
    endcase
//an undefined opcode causes the
//simulator to issue a warning
```

- **unique** asserts that conditions are mutually exclusive and hence it is safe for the expressions to be evaluated in parallel.
  - Compilers may be able to detect potential overlap and issue compile-time warnings.
  - Simulators issue run-time warnings if conditions are evaluated not to be mutually exclusive.
  - Warnings are also issued if no condition is true, or if it is possible that no condition is true.
- If you get such warnings, your combinational logic description is very likely incomplete and hence will not map correctly into hardware!

Decoder example

```
module decoder2to4(
      input logic [1:0] i, output logic
[3:0] outp);

always_comb
begin
  outp = 4'b0000; // default value
  unique case (i)
// unique will force completeness test
     0: outp[0] =  1'b1;
     1: outp[1] =  1'b1;
     2: outp[2] =  1'b1;
     3: outp[3] =  1'b1;
   endcase
end

endmodule
```